

Artificial Neural Networks: Notes

ML A-Z — Part 8

1 The Neuron

An artificial neuron is a loose model of a biological neuron: it takes a set of inputs, combines them, and fires an output signal. For an input vector $\mathbf{x} = (x_1, \dots, x_m)$ (e.g. the independent variables of one observation, typically standardized so all inputs are on comparable scales), a neuron computes

$$z = \sum_{i=1}^m w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b,$$

where w_i is the **weight** on input x_i and b is the **bias** (an always-on input, sometimes written as $w_0 x_0$ with $x_0 = 1$). The weighted sum z is then passed through an **activation function** ϕ to produce the neuron's output $a = \phi(z)$.

Weights are the parameters the network learns: they encode how strongly each input signal influences the neuron downstream.

2 Activation Functions

Without an activation function, a network is just a composition of linear maps, which collapses to a single linear map — no matter how many layers are stacked, the network could only ever learn linear relationships. Activation functions introduce nonlinearity, which is what lets a network approximate arbitrary functions.

- **Threshold function:**

$$\phi(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

A hard on/off gate. Not differentiable at 0, so it cannot be used with gradient-based learning.

- **Sigmoid:**

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

Smooth, squashes z into $(0, 1)$. Useful in the output layer for binary classification, since the output can be read as a probability. Suffers from *vanishing gradients*: for large $|z|$, $\phi'(z) \approx 0$, so learning slows down in saturated neurons.

- **Rectifier (ReLU):**

$$\phi(z) = \max(0, z)$$

The default choice for hidden layers in modern networks. Cheap to compute, and its gradient is either 0 or 1 (no saturation on the positive side), which mitigates vanishing gradients.

- **Hyperbolic tangent (tanh):**

$$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$$

Like sigmoid but zero-centered, which tends to make optimization better behaved.

A common rule of thumb: ReLU in hidden layers, sigmoid (binary) or softmax (multi-class) at the output layer.

3 Network Architecture: Forward Propagation

A fully-connected feed-forward network arranges neurons in layers: an input layer, one or more **hidden layers**, and an output layer. Every neuron in layer ℓ receives, as its inputs, the outputs of every neuron in layer $\ell - 1$.

For layer ℓ with weight matrix $W^{(\ell)}$ and bias vector $\mathbf{b}^{(\ell)}$,

$$\mathbf{z}^{(\ell)} = W^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)}),$$

with $\mathbf{a}^{(0)} = \mathbf{x}$ (the raw input). Applying this layer-by-layer from input to output is **forward propagation**: it turns an input observation into a prediction $\hat{y} = \mathbf{a}^{(L)}$ for a network of L layers.

Intuitively, each hidden layer re-represents the data in a new feature space; deeper layers can combine the simple patterns detected by earlier layers into more complex, abstract features.

4 Cost Function

Learning means choosing weights and biases so that the network's predictions $\hat{y}$ are close to the true targets y . This is measured with a **cost function** (loss function). A standard choice for regression is squared error,

$$C = \frac{1}{2} (\hat{y} - y)^2,$$

summed or averaged over all training examples. (For classification, cross-entropy is typically used instead, since it pairs naturally with a sigmoid/softmax output and produces better gradients when predictions are confidently wrong.)

Training is the optimization problem of finding the weights $\mathbf{w}$ that minimize $C(\mathbf{w})$ over the training set.

5 Gradient Descent

Since C is generally a nonconvex, high-dimensional function of every weight and bias in the network, it cannot be minimized in closed form. **Gradient descent** minimizes it iteratively: at each step, nudge every parameter in the direction that decreases C fastest, i.e. opposite its gradient,

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} C,$$

where η is the **learning rate**.

5.1 Why the Negative Gradient Decreases the Cost

A first-order Taylor expansion of C around the current weights $\mathbf{w}$ justifies the update. For a small step $\Delta\mathbf{w}$,

$$C(\mathbf{w} + \Delta\mathbf{w}) \approx C(\mathbf{w}) + \nabla_{\mathbf{w}}C(\mathbf{w})^\top \Delta\mathbf{w}.$$

Choosing $\Delta\mathbf{w} = -\eta \nabla_{\mathbf{w}}C(\mathbf{w})$ for small $\eta > 0$ gives

$$C(\mathbf{w} + \Delta\mathbf{w}) \approx C(\mathbf{w}) - \eta \|\nabla_{\mathbf{w}}C(\mathbf{w})\|^2 \leq C(\mathbf{w}),$$

since a squared norm is never negative. So, at least locally, stepping opposite the gradient is guaranteed not to increase the cost — this is the entire justification for the algorithm, and it is why any *other* direction (not just the steepest one) would still decrease C as long as it has positive dot product with $-\nabla_{\mathbf{w}}C$.

5.2 Choosing the Learning Rate

The approximation above only holds for small η , so η is not a free parameter to set arbitrarily large:

- **Too large:** the linear approximation breaks down, the step overshoots the minimum, and C can oscillate or diverge entirely instead of decreasing.
- **Too small:** each step barely moves $\mathbf{w}$, so training converges correctly but impractically slowly, and is more likely to stall in a shallow local minimum or saddle point before making meaningful progress.

In practice η is often not held constant: a **learning rate schedule** shrinks η over training (e.g. by a fixed decay factor every few epochs, or on a cosine schedule), taking large, fast steps early and smaller, more precise steps as training approaches a minimum.

5.3 A Worked Example

To make the update rule concrete, take a single-parameter toy cost $C(w) = (w - 3)^2$, so $\nabla_w C = 2(w - 3)$ and the true minimum is $w^* = 3$. Starting at $w_0 = 0$ with $\eta = 0.1$:

$$\begin{aligned}w_1 &= w_0 - \eta \cdot 2(w_0 - 3) = 0 - 0.1(-6) = 0.6, \\w_2 &= w_1 - \eta \cdot 2(w_1 - 3) = 0.6 - 0.1(-4.8) = 1.08, \\w_3 &= w_2 - \eta \cdot 2(w_2 - 3) = 1.08 - 0.1(-3.84) = 1.464.\end{aligned}$$

Each step covers a shrinking fraction of the remaining distance to $w^* = 3$, since the gradient itself shrinks as w approaches the minimum — this geometric-decay behavior is typical of gradient descent on a smooth, well-conditioned cost near its minimum. With $\eta = 1.1$ instead, the same recursion gives $w_1 = 6.6$, $w_2 = -0.6$, growing in magnitude on each step: a concrete instance of the “too large η diverges” failure mode above.

5.4 Batch, Stochastic, and Mini-Batch Gradient Descent

The update rule $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} C$ says nothing about *how much data* is used to compute $\nabla_{\mathbf{w}} C$ before each step. That choice defines three variants, writing C_i for the per-example cost of training example i out of n total, so $C = \frac{1}{n} \sum_{i=1}^n C_i$:

- **Batch gradient descent** computes the exact gradient using the *entire* training set before making one update,

$$\nabla_{\mathbf{w}} C = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}} C_i.$$

This gives an accurate descent direction, but each step costs a full pass over the data, and because C is generally nonconvex, batch GD follows the same direction deterministically every time and can get permanently stuck in a local minimum.

- **Stochastic gradient descent (SGD)** updates the weights after *each* individual training example i , using that example's (noisy) gradient in place of the true one,

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} C_i.$$

This single-example gradient is an unbiased estimate of the true gradient ($\mathbb{E}_i[\nabla_{\mathbf{w}} C_i] = \nabla_{\mathbf{w}} C$ when i is drawn uniformly at random) but a high-variance one. SGD is also better suited to large or streaming datasets, since it never needs the whole dataset in memory to take a step.

- **Mini-batch gradient descent** is the practical middle ground used by most deep learning frameworks: compute the gradient over a small batch B of size m (e.g. 32–256 examples) at a time,

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{m} \sum_{i \in B} \nabla_{\mathbf{w}} C_i.$$

This retains much of SGD's escape-local-minima behavior while reducing gradient variance relative to single-example SGD, and it vectorizes cleanly onto GPU hardware for speed.

	Batch GD	SGD	Mini-batch GD
Examples per update	n (all)	1	$m \ll n$
Gradient variance	none (exact)	high	moderate
Cost per update	high	very low	low
Updates per epoch	1	n	n/m

5.5 Why Does SGD's Noise Help?

It seems counterintuitive that using a *worse* (noisier) estimate of the gradient could help optimization, but the noise plays two distinct useful roles:

- **Escaping local minima and saddle points.** Batch GD's direction is a deterministic function of $\mathbf{w}$, so once it reaches a point with zero gradient it stops there, whether that point is a good minimum or not. SGD's per-step direction varies randomly around the true gradient, so even at a point where the *true* gradient is zero, the noisy single-example gradient usually is not — giving the optimizer a chance to keep moving through shallow local minima and saddle points instead of stalling in them.

- **Implicit regularization.** Because SGD never fully converges to any single sharp minimum of a mini-batch's cost surface (the next batch immediately pulls it in a slightly different direction), it tends to settle in wider, flatter minima of the true cost surface. Flatter minima generally generalize better to unseen data than sharp ones, so SGD's noise acts as a mild, essentially free regularizer on top of being computationally cheaper.

6 Backpropagation

Backpropagation is the algorithm that computes $\nabla_{\mathbf{w}} C$ efficiently, layer by layer, using the chain rule — rather than perturbing each weight independently (which would cost one full forward pass per weight).

6.1 Setup

Let layer ℓ have weighted input $\mathbf{z}^{(\ell)}$ and activation $\mathbf{a}^{(\ell)} = \phi(\mathbf{z}^{(\ell)})$, for $\ell = 1, \dots, L$, with $\mathbf{a}^{(L)} = \hat{y}$ the network output. Define the **error** at layer ℓ as

$$\boldsymbol{\delta}^{(\ell)} = \frac{\partial C}{\partial \mathbf{z}^{(\ell)}}.$$

6.2 Output layer error

By the chain rule,

$$\boldsymbol{\delta}^{(L)} = \frac{\partial C}{\partial \mathbf{a}^{(L)}} \odot \phi'(\mathbf{z}^{(L)}),$$

where $\odot$ is the elementwise product. For squared error, $\partial C / \partial \mathbf{a}^{(L)} = \hat{y} - y$.

6.3 Propagating the error backward

The error at layer ℓ depends on the error one layer ahead of it:

$$\boldsymbol{\delta}^{(\ell)} = \left(\left(W^{(\ell+1)} \right)^\top \boldsymbol{\delta}^{(\ell+1)} \right) \odot \phi'(\mathbf{z}^{(\ell)}).$$

This is the key equation: it lets $\boldsymbol{\delta}^{(\ell)}$ be computed from $\boldsymbol{\delta}^{(\ell+1)}$, moving backward from the output layer to the input layer — hence *backpropagation*. It is exactly the chain rule applied once per layer, reusing work already done for the layer ahead instead of recomputing it from scratch.

6.4 Gradients from the error

Once every $\boldsymbol{\delta}^{(\ell)}$ is known, the gradients needed for the weight update fall out directly:

$$\frac{\partial C}{\partial W^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} \left(\mathbf{a}^{(\ell-1)} \right)^\top, \quad \frac{\partial C}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}.$$

6.5 Algorithm

Repeating this over all training examples, for many passes (**epochs**), is how the network learns: each epoch, weights move a little further in the direction that reduces the cost function.

Algorithm 1 Training one example with backpropagation + SGD

```
1: Forward pass: compute  $\mathbf{z}^{(\ell)}, \mathbf{a}^{(\ell)}$  for  $\ell = 1, \dots, L$ 
2: Output error:  $\delta^{(L)} \leftarrow (\hat{y} - y) \odot \phi'(\mathbf{z}^{(L)})$ 
3: for  $\ell = L - 1, \dots, 1$  do
4:    $\delta^{(\ell)} \leftarrow \left( (W^{(\ell+1)})^\top \delta^{(\ell+1)} \right) \odot \phi'(\mathbf{z}^{(\ell)})$ 
5: end for
6: for  $\ell = 1, \dots, L$  do
7:    $W^{(\ell)} \leftarrow W^{(\ell)} - \eta \delta^{(\ell)} (\mathbf{a}^{(\ell-1)})^\top$ 
8:    $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \delta^{(\ell)}$ 
9: end for
```

7 Key Takeaways

- Activation functions are what give a network the capacity to learn nonlinear relationships; ReLU is the standard hidden-layer default, sigmoid/softmax the standard output-layer choice.
- Forward propagation produces a prediction; backpropagation produces the gradient of the cost with respect to every weight, by reusing per-layer error terms via the chain rule instead of recomputing each partial derivative independently.
- SGD (or mini-batch GD) trades a noisier gradient estimate for much cheaper, much more frequent updates, and that noise is what helps escape local minima in the nonconvex cost surface.
- The learning rate η controls the step size and is the main knob balancing convergence speed against stability.